1. Control Structures

In a program, statements normally execute one after another in order.

But sometimes we want to:

- Make decisions
- Repeat actions
- Choose different paths

These are handled using control structures.

Control structures are language constructs that control the flow of execution of a program. They decide:

- which statements run
- how many times they run
- in what order they run

Control structures help to:

- Solve real-world problems logically
- Reduce repeated code
- Make programs flexible
- Implement conditions and loops
- Control program behavior

Main Types of Control Structures

Control structures are usually classified into three major types:

1. Sequence
2. Selection (Decision-Making)
3. Iteration (Looping / Repetition)

1. **Sequence Control Structure**

   In sequence, statements are executed one by one in the order they appear.

Example :

```
read a
read b
sum = a + b
print sum
```

Features:

- Default execution style (Top to bottom execution)
- No conditions
- No repetition
- Simple and straightforward

---

2. **Selection Control Structure**

   Selection allows the program to choose one path based on a condition.

   Also called decision-making statements.

   Types of control statements

   - if
   - if–else
   - nested if
   - switch or case

   Example:

```
if (marks >= 50)
  print "Pass"
else
  print "Fail"
```

   Purpose

   - Tests conditions
   - Executes different blocks
   - Implements logic

3. **Iteration Control Structure:**
   Iteration means repeating a set of statements while a condition is true. Also called loops.

   Common Loop Types

   - while
   - do–while
   - for

   Example:

   ```
   for i = 1 to 5
     print i
   ```

   Purpose

   - Avoids duplicate code
   - Handles repeated calculations
   - Saves time and space
     Assignment Statements

---

2. Assignment Statement

- An assignment statement gives a value to a variable.
- An assignment statement stores the result of an expression into a variable.

General Form

variable = expression

Example:

x = 10

sum = a + b

avg = total / n

Parts of an Assignment Statement

- **Left side** → variable (destination)
- **Right side** → expression (value to be stored)

Example:

result = x * y + 5

- result = variable
- x * y + 5 = expression

Characteristics

- Evaluates the right-hand side first
- Stores the value in the left-hand variable
- Overwrites the old value
- Must match type rules of the language

Types of Assignment

- **Simple Assignment**

  a = 5

- **Compound Assignment**

  Short form:

  a += 3 (means a = a + 3)

  b *= 2 (means b = b * 2)

- **Multiple Assignment (in some languages)**

  x = y = 0

Type Checking in Assignment

Languages ensure:

- Right-side value fits left-side variable type
- Sometimes automatic conversion happens

Example:

```
int a;
a = 3.5;   // may become 3
```

Uses

- Store results
- Update variables
- Control program flow
- Perform calculations

---

3. Boolean Expressions

A **Boolean expression** is an expression that gives either:

- **true**
- **false**

It is mainly used in:

- decision statements (if)

- loops (while, for)

Relational Operators

Used to compare values:

- > greater than
- < less than
- >= greater or equal
- <= less or equal
- == equal
- != not equal

Example:

a > b

x == 10

marks >= 50

Logical Operators

Used to combine conditions:

- AND (&&)
- OR (||)
- NOT (!)

Examples:

a > 0 && b > 0

x < 5 || y > 10

!(a == b)

Boolean Variables

Variables that store true/false:

bool passed = true;

Use in Control Structures

- **In Selection:**

```
if (age >= 18)
   print "Eligible";
else
   print “Not Eligible”;
```

- **In Loops:**

```
while (count < 10)
   count = count + 1;
```

Operator Precedence in Boolean Expressions

Common order:

1. NOT
2. AND
3. OR

Example:

a > 0 || b > 0 && c > 0

b > 0 && c > 0 is evaluated first.

---

4. Arithmetic Expressions

An **arithmetic expression** is a combination of:

- **numbers or variables** (operands)
- **arithmetic operators**

used to perform calculations.

Parts of an Arithmetic Expression

1. **Operands**

Values or variables used in calculation.
Examples: 5, x, total

2. **Operators**

Symbols that perform arithmetic operations:

- + → addition
- - → subtraction
- * → multiplication
- / → division
- % → remainder

Examples

a + b

x * y

(p + q) / r

m % 2

Operator Precedence

When several operators appear, some are evaluated first.

Order:

1. Brackets ()
2. *, /, %
3. +, -

Example:

a + b * c

Here, b * c is evaluated first.

Unary and Binary Operators

1. **Unary Operator**

   Works on one operand.
   Examples: -x, +a

2. **Binary Operator**

   Works on two operands.
   Examples: a + b, x * y

Type Issues in Arithmetic Expressions

Operands may be of different types:

- integer + integer → integer
- integer + float → float

Languages define rules for:

- type conversion
- coercion
- compatibility